

머신러닝

나이프 베이지안

Chapter 10



Prof. Yang



한국성서대학교
KOREAN BIBLE UNIVERSITY

목 차

CONTENTS

- 01 베이지 정리 이해
- 02 베이지 분류기 구현
- 03 나이브 베이지안 분류기

01

베이지즈 정리의 이해



확률의 표현

- 이산형 값의 확률

$$P(X) = \frac{\text{count}(\text{Event}_x)}{\text{count}(\text{Event}_{\text{allevent}})}$$

- 연속형 값의 확률 : 해당 데이터를 적절히 표현하는 함수를 생성한 후 해당 함수의 적분 값을 취함

- 특정 위치의 값은 없고 적분 값을 취해 구간의 확률을 계산

$$P(-\infty < x < \infty) \int_{-\infty}^{\infty} f(x) dx = 1$$

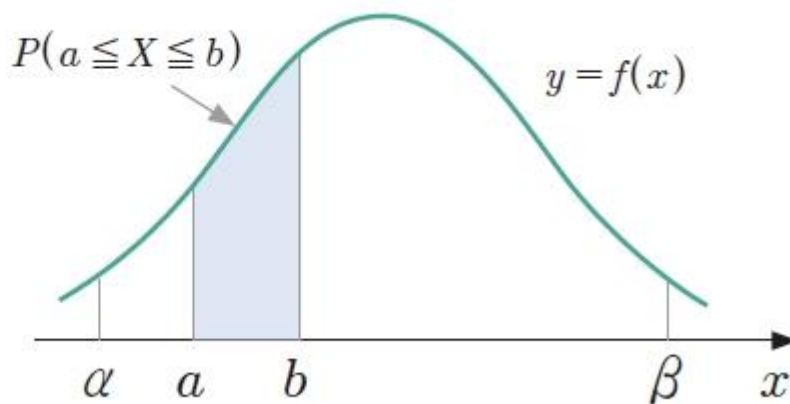


그림 11-2 연속형 값에 대한 확률 분포



확률의 기본 성질

- 확률은 모든 사건에 대해 반드시 0에서 1사이의 값을 가짐

$$0 \leq P(X) \leq 1$$

- 각 사건들이 서로 관계가 없는 경우, 즉 각 사건들이 일어날 확률이 다른 사건이 일어날 확률에 영향을 미치지 않을 때 각 사건들이 '독립'되었다고 정의

$$P(S) = \sum_{i=1}^N P(E_i) = 1$$



조건부 확률

- 조건부 확률(conditional probability) : 어떤 사건이 일어난다고 가정했을 때 다른 사건이 일어날 확률
 - $P(A|B)$: B라는 사건이 발생했을 때 A와 B 사건의 교집합이 발생할 확률

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

- 해당 사건의 $(A|B)$ 는 B의 상황에서 A를 만족하는 값을 찾으면 되므로 $\{1, 3\}$

$$A = \{x | x \text{는 홀수}\}, \quad B = \{x | x \text{는 4보다 작은 수}\}$$

$$\frac{\cancel{2} \cancel{6}}{\cancel{3} \cancel{6}} = \frac{2}{3}$$



베이즈 정리

- 베이즈 정리(Bayes' theorem) : 두 확률 변수의 사전확률과 사후확률 사이의 관계를 나타내는 정리
- 베이즈 정리의 전제 : 객관적인 확률이 존재하지 않고 이전 사건으로 인해 확률이 지속적으로 업데이트됨
 - 베이즈주의자는 실제로 뒤집어 카드가 나온 확률을 기반으로 실행할 때마다 계속하여 확률들을 업데이트
=> 경험을 반영한 실제에 가까운 값



- 빈도주의자는 다음 확률을 계속하여 실행할 경우 $\frac{1}{3}$ 에 수렴할 것이므로 $\frac{1}{3}$ 로 간주



베이즈 정리

- H는 가설, D는 데이터일 때 $P(H|D)$ 는 사후 확률

$$P(H|D) = \frac{P(H)P(D|H)}{P(D)} = \frac{\cancel{P(H)} \times \frac{P(D \cap H)}{\cancel{P(H)}}}{P(D)} = \frac{P(D \cap H)}{P(D)}$$

조건부 확률

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

- 예

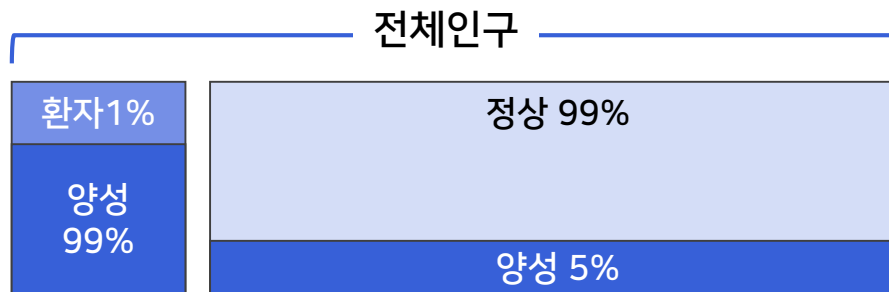
- 전체 인구 중 질병을 가진 사람: 1%

- 이 질병 검사 정확도:

질병이 있는 사람은 99% 확률로 양성

질병이 없는 사람도 5% 확률로 양성(위양성 음성임에도 불구하고 양성으로 나타나는 현상)

- 이 경우 나의 검사 결과 양성이 나왔을 때, 실제로 질병일 확률은?



$$P(\text{질병}|\text{양성}) = \frac{0.01 \times 0.99}{0.01 \times 0.99 + 0.99 \times 0.05} = \frac{0.0099}{0.0594} \approx 0.1667 (\text{약 } 16.7\%)$$

- 결론: 검사 결과가 양성이라도 실제로 질병일 확률은 약 16.7%!

02 베이지스 분류기 구현



베이지 분류기 만들기

- 메일에 비아그라(viagra)라는 단어가 들어가면 어느 정도의 확률로 스팸메일인지 판단하는 베이지 분류기

$$P(H|D) = \frac{P(H)P(D|H)}{P(D)}$$

➡

$$P(spam|viagra) = \frac{P(spam)P(viagra|spam)}{P(viagra)}$$

표 11-1 비아그라와 스팸메일을 나타내는 임의의 데이터

number	viagra	spam	number	viagra	spam
1	1	1	11	1	0
2	0	0	12	0	0
3	0	0	13	0	0
4	0	0	14	1	0
5	0	0	15	0	0
6	0	0	16	0	0
7	0	1	17	0	0
8	0	0	18	0	1
9	1	1	19	0	1
10	1	0	20	1	1

$$P(spam|viagra) = \frac{\frac{6}{20} \times \frac{3}{6}}{\frac{6}{20}}$$



베이지스 분류기 코드

■ 데이터 준비

```
import pandas as pd
import numpy as np

viagra_spam = {'viagra':[1,0,0,0,0,0,0,0,1,1,1,0,0,1,0,0,0,0,0,1],
               'spam':[1,0,0,0,0,0,1,0,1,0,0,0,0,0,0,0,0,0,1,1,1]}
np_data = pd.DataFrame(viagra_spam).values
np_data
```

```
array([[1, 1],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 1],
       [0, 0],
       [1, 1],
       [1, 0],
       [1, 0],
       [0, 0],
       [0, 0],
       [1, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 1],
       [0, 1],
       [1, 1]], dtype=int64)
```



베이지 분류기 코드

- 각 확률 구하기

```
#P(viagra)
p_v = sum(np_data[:, 0] == 1) / len(np_data)

#P(spam)
p_s = sum(np_data[:, 1] == 1) / len(np_data)

#P(viagra&spam)
p_vs = sum((np_data[:, 0] == 1) & (np_data[:, 1] == 1)) / len(np_data)

#P(~viagra&spam)
p_n_vs = sum((np_data[:, 0] == 0) & (np_data[:, 1] == 1)) / len(np_data)

p_v, p_s, p_vs, p_n_vs

(0.3, 0.3, 0.15, 0.15)
```



베이지 분류기 코드

- 메일에 비아그라(viagra) 있을 경우 스팸메일일 확률 $P(spam|viagra)$

$$p_s * (p_{vs} / p_s) / p_v$$

0.5

- 메일에 비아그라(viagra)가 없는데 스팸메일일 확률 $P(spam|\sim viagra)$

$$p_s * (p_{vs} / p_s) / (1-p_v)$$

0.2142857142857143

- 분석
 - 'viagra'라는 단어가 포함되었을 때 스팸메일일 확률(0.5)은 'viagra'라는 단어가 포함되지 않았을 때 스팸 메일일 확률(약 0.21)보다 높음
 - 'viagra'라는 단어가 있으면 스팸메일로 분류하는 것이 합리적

03

나이프 베이지안 분류기



나이브 베이지안 분류기 (Naive Bayesian Classifier)

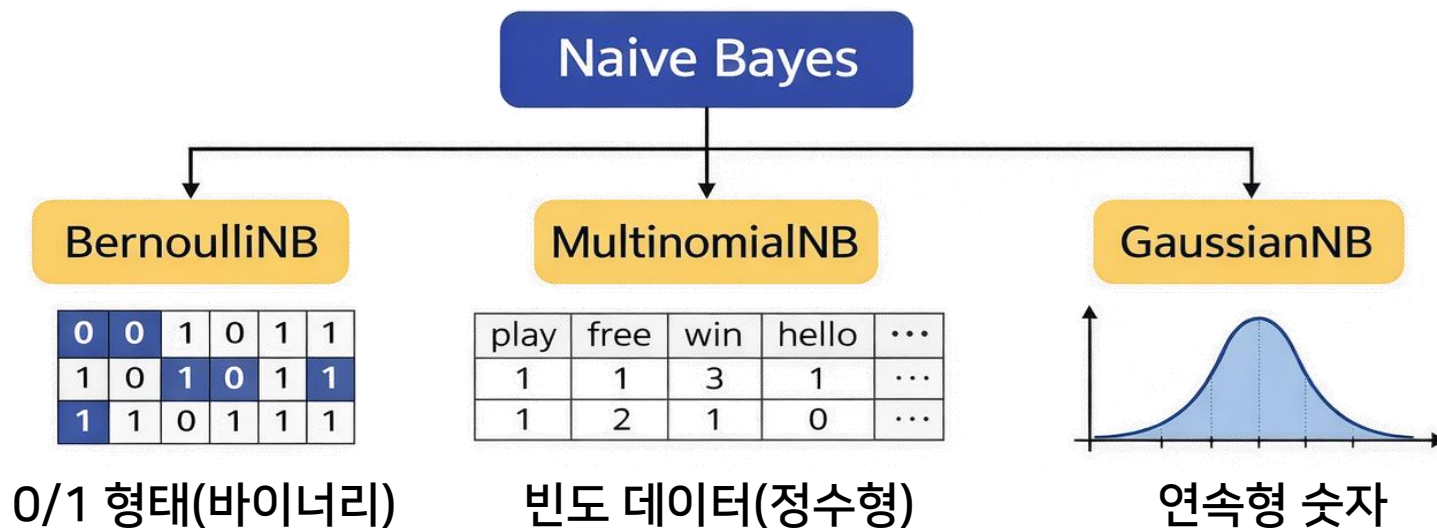
- 여러 개의 피쳐를 사용하여 클래스(label)를 예측하는 확률 기반 분류기

$$P(Y_c | X_1, \dots, X_n) = \frac{P(Y_c) \prod_{i=1}^n P(X_i | Y_c)}{\prod_{i=1}^n P(X_i)}$$

$\prod_{k=1}^n a_k = a_1 \times a_2 \times a_3 \times \dots \times a_n$
 Y_c is a label

- 사이킷런 기반 나이브 베이지안 종류: feature 형태에 따라 모델 선택

- 아래 모델 모두 다중 분류는 가능



베이지안 계열 역시 분류 모델이므로
혼동행렬(confusion matrix)와
이를 기반으로 한 정확도, 재현율 등의 지표를
이용하여 모델 성능을 평가할 수 있음

베르누이 나이브 베이지안(Bernoulli Naive Bayes)

- : 0/1 이진 feature 데이터를 사용하는 나이브 베이지안 모델
- fraud 데이터 셋 활용

열 이름	데이터	의미
ID	숫자	인덱스
History	current	신용 계약이 현재 진행 중
	none	신용 기록이 없음
	paid	신용 계약을 완전히 상환
coApplicant	coApplicant	대출 신청 시 신청자 외에 다른 사람이 공동 신청자로 등록
	guarantor	보증인이 대출 신청에 포함
	none	보증인 없이 신청자 혼자서 대출
Accommodation	free	무료로 거주
	own	집 소유
	rent	임대 주택
fraud	1	사기꾼 O
	0	사기꾼 X



베르누이 나이브 베이지안

- 데이터 읽기

```
import pandas as pd
import numpy as np

data_url = "https://raw.githubusercontent.com/prof-dana/Lecture-Code/refs/heads/main/MachineLearning/data/fraud.csv"
df= pd.read_csv(data_url)
df.head()
```

	ID	History	CoApplicant	Accommodation	Fraud
0	1	current	none	own	True
1	2	paid	none	own	False
2	3	paid	none	own	False
3	4	paid	guarantor	rent	True
4	5	arrears	none	own	False



베르누이 나이브 베이지안

■ 전처리

- ID열 제거
- Y 값을 따로 빼내고 X 데이터들을 원핫인코딩으로 처리

```
df.drop(columns=["ID"],inplace=True)
Y_data = df.pop("Fraud") #Fraud열은 빼서 Y_data에 저장
Y_data = Y_data.values #넘파이 array로 바꿈(열 이름X)
x_df = pd.get_dummies(df)
x_df.head(10).T
```

	0	1	2	3	4	5	6	7	8	9
History_arrears	False	False	False	False	True	True	False	True	False	False
History_current	True	False	False	False	False	False	True	False	True	False
History_none	False	False	False	False	False	False	False	False	False	True
History_paid	False	True	True	True	False	False	False	False	False	False
CoApplicant_coapplicant	False	False	False	False	False	False	False	False	False	False
CoApplicant_guarantor	False	False	False	True	False	False	False	False	False	False
CoApplicant_none	True	True	True	False	True	True	True	True	True	True
Accommodation_free	False	False	False	False	False	False	False	False	False	False
Accommodation_own	True	True	True	False	True	True	True	True	False	True
Accommodation_rent	False	False	False	True	False	False	False	False	True	False

현재 데이터셋의 크기가 작아 훈련 데이터와
테스트 데이터를 별도로 분리하지 않음

```
x_data = x_df.values #넘파이 배열로 변환
x_data[:5]
```

```
array([[False,  True, False, False, False, False,  True, False,  True,
        False],
       [False, False, False,  True, False, False,  True, False,  True,
        False],
       [False, False, False,  True, False, False,  True, False,  True,
        False],
       [False, False, False,  True, False,  True, False, False, False,
        True],
       [ True, False, False, False, False, False,  True, False,  True,
        False]])
```

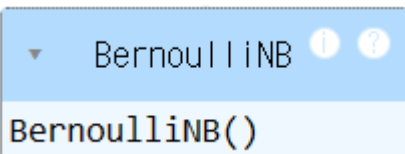


베르누이 나이브 베이지안

■ 학습하기

```
from sklearn.naive_bayes import BernoulliNB
```

```
model = BernoulliNB()  
model.fit(x_data, Y_data)
```



■ 예측 확률 확인

```
y_prob = model.predict_proba(x_data)
```

```
print(y_prob)
```

```
[[0.71084848 0.28915152]  
 [0.88059963 0.11940037]  
 [0.88059963 0.11940037]  
 [0.17911719 0.82088281]  
 [0.92657726 0.07342274]  
 [0.92657726 0.07342274]  
 [0.71084848 0.28915152]  
 [0.92657726 0.07342274]  
 [0.34439038 0.65560962]  
 [0.51962189 0.48037811]  
 [0.73328558 0.26671442]  
 [0.71084848 0.28915152]  
 [0.34439038 0.65560962]  
 [0.88059963 0.11940037]  
 [0.92657726 0.07342274]  
 [0.71084848 0.28915152]  
 [0.75097402 0.24902598]  
 [0.87517127 0.12482873]  
 [0.92657726 0.07342274]  
 [0.88059963 0.11940037]]
```



베르누이 나이브 베이지안

- 예측값 확인

```
y_pred = model.predict(x_data)
```

```
print(y_pred)
```

```
[False False False  True False False False False  True False False False
  True False False False False False False False]
```

- 간단하게 정확도 확인

```
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(Y_data, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
Accuracy: 0.75
```



다항 나이브 베이지안 분류기(MultinomialNB)

- : feature가 빈도(count) 형태의 데이터를 사용할 때 적용되는 나이브 베이지안 분류 모델
 - 하나의 문장이 있을 때 이 문장을 "sports"와 "politics"로 나누는 분류기 만들기

```
text_example = [ "A great game game",
                  "The team played a great game",
                  "The match was exciting",
                  "A clean game with great players",
                  "The team won the match",
                  "The player scored a goal",
                  "It was a close game",
                  "The championship match was amazing",
                  "The team celebrated the victory",
                  "A fantastic game by the players",
                  "The election results were announced",
                  "The election campaign was intense",
                  "The president won the election",
                  "The government announced new policy",
                  "The parliament passed the law",
                  "The political debate was heated",
                  "The candidate started the campaign",
                  "The voters supported the candidate",
                  "The election vote counting began",
                  "The government election strategy changed"]

y = ["sports","sports","sports","sports","sports",
     "sports","sports","sports","sports","sports",
     "politics","politics","politics","politics","politics",
     "politics","politics","politics","politics","politics"]
```



다항 나이프 베이지안 분류기

- 전처리는 BoW(Bag of Words) 사용
 - : 단어별로 인덱스가 부여되어 있을 때 한 문장 또는 한 문서에 대한 벡터를 표현하는 기법
 - 전체 문서에 등장하는 모든 단어에 인덱스를 부여한 뒤, 각 문서를 단어 출현 빈도 기반 벡터로 표현
 - 등장하지 않은 단어는 0으로 표시함

표 11-2 BoW 기법으로 표현된 데이터

	are	call	from	hello	home	how	me	money	now	tomorrow	win	you
0	1	0	0	1	0	1	0	0	0	0	0	1
1	0	0	1	0	1	0	0	1	0	0	2	0
2	0	1	0	0	0	0	1	0	1	0	0	0
3	0	1	0	1	0	0	0	0	0	1	0	1



다항 나이브 베이지안 분류기

- 전처리는 BoW(Bag of Words) 사용

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
vectorizer = CountVectorizer()
```

```
X = vectorizer.fit_transform(text_example)
```

```
print(vectorizer.get_feature_names_out())#열 이름
```

```
print(X.toarray()[:5])
```

```
['amazing' 'announced' 'began' 'by' 'campaign' 'candidate' 'celebrated'
 'championship' 'changed' 'clean' 'close' 'counting' 'debate' 'election'
 'exciting' 'fantastic' 'game' 'goal' 'government' 'great' 'heated'
 'intense' 'it' 'law' 'match' 'new' 'parliament' 'passed' 'played'
 'player' 'players' 'policy' 'political' 'president' 'results' 'scored'
 'started' 'strategy' 'supported' 'team' 'the' 'victory' 'vote' 'voters'
 'was' 'were' 'with' 'won']
[[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0
  0 0 0 1 1 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
  0 0 0 0 1 0 0 0 1 0 0]
 [0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0
  0 0 0 0 0 0 0 0 0 0 1]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
  0 0 0 1 2 0 0 0 0 0 1]]
```

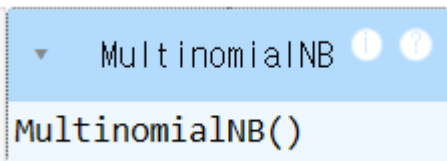


다항 나이브 베이지안 분류기

■ 학습하기

```
from sklearn.naive_bayes import MultinomialNB
```

```
model = MultinomialNB()  
model.fit(X, y)
```



■ 테스트 데이터로 예측하기

```
test = ["the player wins"] # "the government election policy"  
X_test = vectorizer.transform(test) # 변환
```

```
print(model.predict_proba(X_test))  
print(model.classes_) # 라벨 이름  
print(model.predict(X_test))
```

```
[[0.40313534 0.59686466]]  
['politics' 'sports']  
['sports']
```




가우시안 나이브 베이지안 분류기(GaussianNB)

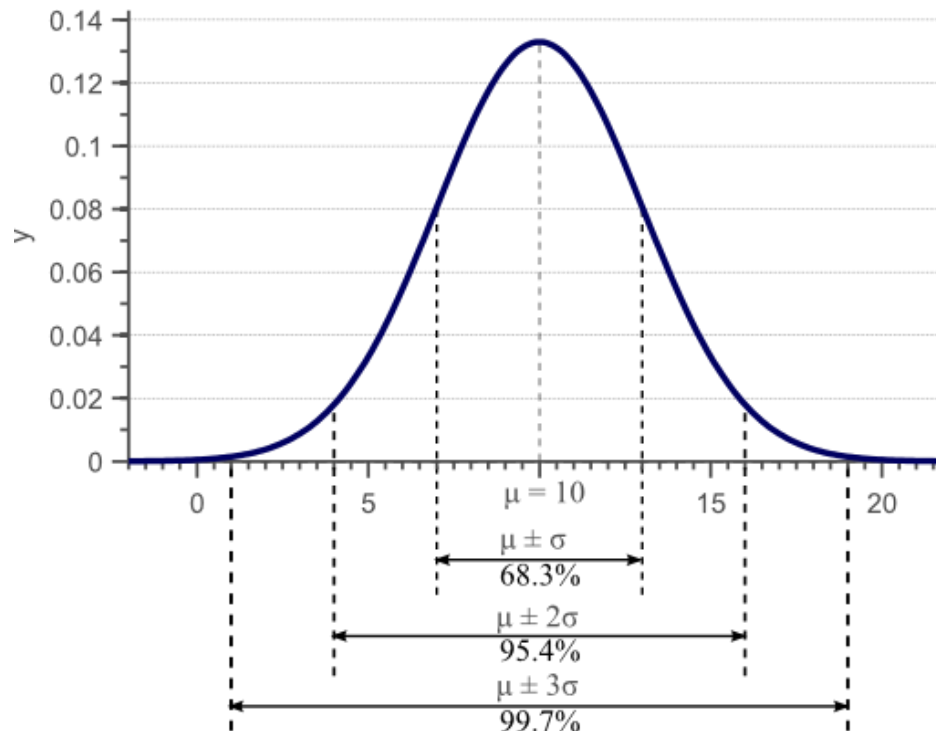
- : 피쳐가 연속형 숫자 데이터일 때, 각 피쳐가 클래스별 정규분포를 따른다고 가정하는 나이브 베이지안 분류
- 확률밀도함수의 값을 이용하여 조건부확률을 계산하고 이를 기반으로 나이브 베이지안(NB)을 구현

- 확률

- 평균 μ
- 표준편차 σ / 분산 σ^2

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

$$P(x_i | Y_c) = \frac{1}{\sqrt{2\pi\sigma_{Y_c}^2}} \exp \left(-\frac{(x_i - \mu_{Y_c})^2}{2\sigma_{Y_c}^2} \right)$$





가우시안 나이브 베이지안 분류기

■ Iris 데이터 셋 준비

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	4.9	3.0	1.4	0.2	setosa
1	4.7	3.2	1.3	0.2	setosa
2	4.6	3.1	1.5	0.2	setosa
3	6.4	3.2	4.5	1.5	versicolor
4	6.9	3.1	4.9	1.5	versicolor
5	5.5	2.3	4.0	1.3	versicolor
6	7.1	3.0	5.9	2.1	virginica
7	6.3	2.9	5.6	1.8	virginica
8	7.6	3.0	6.6	2.1	virginica



```
from sklearn import datasets #내장 데이터셋 사용
from sklearn.naive_bayes import GaussianNB
import pandas as pd

iris = datasets.load_iris() #샘플 데이터 load
df_X=pd.DataFrame(iris.data) # feature
df_Y=pd.DataFrame(iris.target) #label(0,1,2로 표현)
iris.target_names
```

```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```



가우시안 나이브 베이지안 분류기

■ 데이터 분리

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(df_X, df_Y, test_size=0.25)  
X_train.shape, X_test.shape
```

```
((112, 4), (38, 4))
```

■ 학습하기

```
from sklearn.naive_bayes import GaussianNB
```

```
clf = GaussianNB()  
clf.fit(X_train, y_train)
```

```
▼ GaussianNB ⓘ ?  
GaussianNB()
```



가우시안 나이브 베이지안 분류기

■ 예측하기

```
#꽃받침 길이 5.1cm, 꽃받침 너비 3.5cm, 꽃잎 길이 1.4cm, 꽃잎 너비 0.2cm
test = [[5.1, 3.5, 1.4, 0.2]]
idx = clf.predict(test)[0]
print(iris.target_names[idx])
```

setosa

■ 평가하기

```
from sklearn.metrics import confusion_matrix
```

```
y_pred = clf.predict(X_test)
confusion_matrix(y_test, y_pred)
```

```
array([[10,  0,  0],
       [ 0, 20,  1],
       [ 0,  1,  6]], dtype=int64)
```

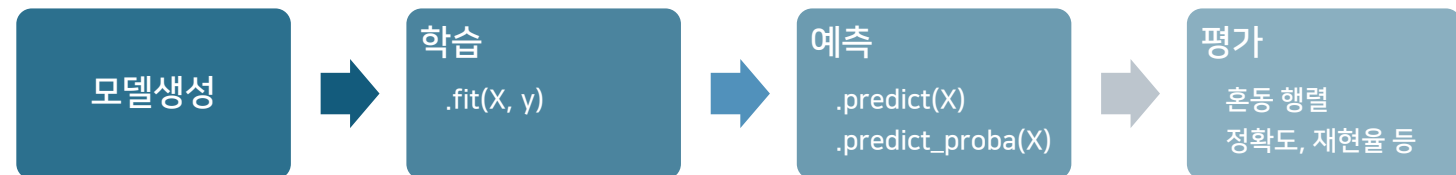
```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	0.95	0.95	0.95	21
2	0.86	0.86	0.86	7
accuracy			0.95	38
macro avg	0.94	0.94	0.94	38
weighted avg	0.95	0.95	0.95	38



SUMMARY

- 베이즈 정리 : 조건부 확률을 이용하여 사건의 사후 확률을 계산하는 확률 이론
 - 구현 : 입력 데이터로 각 클래스에 속할 확률을 계산하고 가장 높은 확률을 가진 클래스로 분류
- 나이브 베이지안 분류기 : 각 feature가 서로 독립이라는 가정을 두고 베이즈 정리를 단순화하여 분류를 수행하는 방법
 - 베르누이 나이브 베이지안: feature가 이진 데이터(0/1)일 때 사용하는 모델
 - BernoulliNB()
 - 다항 나이브 베이지안: 단어 등장 횟수와 같은 빈도(count) 데이터에 적합한 모델
 - MultinomialNB()
 - 가우시안 나이브 베이지안: feature가 연속형 데이터이며 클래스별 정규분포를 따른다고 가정하는 모델
 - GaussianNB()





Q&A

궁금한 사항은 아래 방법으로 문의 바랍니다.



이메일
dana1112@bible.ac.kr



LMS 쪽지

THANK YOU FOR YOUR ATTENTION!

한국성서대학교 | 인공지능융합학부 | 머신러닝